

25.1 Maximum bipartite matching (revisited)

Section 24.3 demonstrated one way to find a maximum matching in a bipartite graph, by finding a maximum flow. This section provides a more efficient method, the Hopcroft-Karp algorithm, which runs in $O(\sqrt{V}E)$ time. Figure 25.1(a) shows a matching in an undirected bipartite graph. A vertex that has an incident edge in matching M is *matched* under M , and otherwise, it is *unmatched*. A *maximal matching* is a matching M to which no other edges can be added, that is, for every edge $e \in E - M$, the edge set $M \cup \{e\}$ fails to be a matching. A maximum matching is always maximal, but the reverse does not always hold.

Many algorithms to find maximum matchings, the Hopcroft-Karp algorithm included, work by incrementally increasing the size of a matching. Given a matching M in an undirected graph $G = (V, E)$, an *M -alternating path* is a simple path whose edges alternate between being in M and being in $E - M$. Figure 25.1(b) depicts an *M -augmenting path* (sometimes called an augmenting path with respect to M): an M -alternating path whose first and last edges belong to $E - M$. Since an M -augmenting path contains one more edge in $E - M$ than in M , it must consist of an odd number of edges.

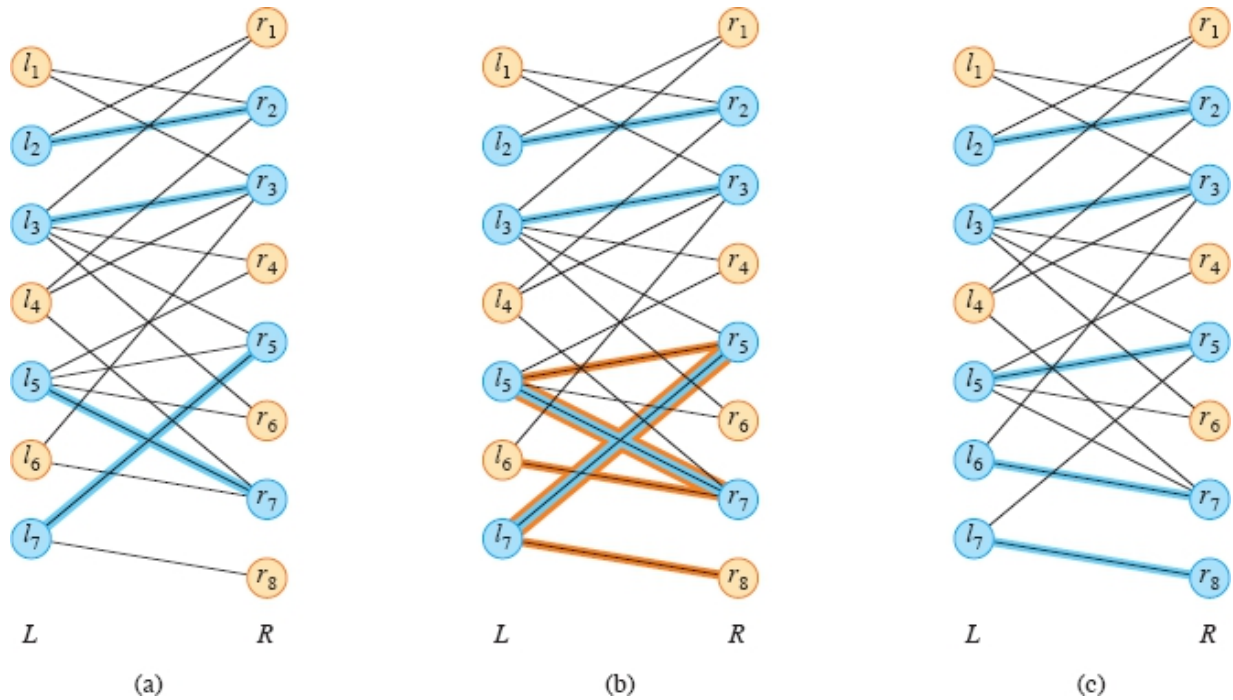


Figure 25.1 A bipartite graph, where $V = L \cup R$, $L = \{l_1, l_2, \dots, l_7\}$, and $R = \{r_1, r_2, \dots, r_8\}$. **(a)** A matching M with cardinality 4, highlighted in blue. Matched vertices are blue, and unmatched vertices are tan. **(b)** The five edges highlighted in orange form an M -augmenting path P going between vertices l_6 and r_8 . **(c)** The set of edges $M' = M \oplus P$ highlighted in blue is a matching containing one more edge than M and adding l_6 and r_8 to the matched vertices. This matching is not a maximum matching (see Exercise 25.1-1).

Figure 25.1(c) demonstrates the following lemma, which shows that by removing from matching M the edges in an M -augmenting path that belong to M and adding to M the edges in the M -augmenting path that are not in M , the result is a new matching with one more edge than M . Since a matching is a set of edges, the lemma relies on the notion of the *symmetric difference* of two sets: $X \oplus Y = (X - Y) \cup (Y - X)$, that is, the elements that belong to X or Y , but not both. Alternatively, you can think of $X \oplus Y$ as $(X \cup Y) - (X \cap Y)$. The operator $\oplus$ is commutative and associative. Furthermore, $X \oplus X = \emptyset$ and $X \oplus \emptyset = \emptyset \oplus X = X$ for any set X , so that the empty set is the identity for $\oplus$.

Lemma 25.1

Let M be a matching in any undirected graph $G = (V, E)$, and let P be an M -augmenting path. Then the set of edges $M' = M \oplus P$ is also a

matching in G with $|M'| = |M| + 1$.

Proof Let P contain q edges, so that $\lceil q/2 \rceil$ edges belong to $E - M$ and $\lfloor q/2 \rfloor$ edges belong to M , and let these q edges be $(v_1, v_2), (v_2, v_3), \dots, (v_q, v_{q+1})$. Because P is an M -augmenting path, vertices v_1 and v_{q+1} are unmatched under M and all other vertices in P are matched. Edges $(v_1, v_2), (v_3, v_4), \dots, (v_q, v_{q+1})$ belong to $E - M$, and edges $(v_2, v_3), (v_4, v_5), \dots, (v_{q-1}, v_q)$ belong to M . The symmetric difference $M' = M \oplus P$ reverses these roles, so that edges $(v_1, v_2), (v_3, v_4), \dots, (v_q, v_{q+1})$ belong to M' and $(v_2, v_3), (v_4, v_5), \dots, (v_{q-1}, v_q)$ belong to $E - M'$. Each vertex $v_1, v_2, \dots, v_q, v_{q+1}$ is matched under M' , which gains one additional edge relative to M , and no other vertices or edges in G are affected by the change from M to M' . Hence, M' is a matching in G , and $|M'| = |M| + 1$. ■

Since taking the symmetric difference of a matching M with an M -augmenting path increases the size of the matching by 1, the following corollary shows that taking the symmetric difference of M with k vertex-disjoint M -augmenting paths increases the size of the matching by k .

Corollary 25.2

Let M be a matching in any undirected graph $G = (V, E)$ and $P_1, P_2, \dots, P_k$ be vertex-disjoint M -augmenting paths. Then the set of edges $M' = M \oplus (P_1 \cup P_2 \cup \dots \cup P_k)$ is a matching in G with $|M'| = |M| + k$.

Proof Since the M -augmenting paths $P_1, P_2, \dots, P_k$ are vertex-disjoint, we have that $P_1 \cup P_2 \cup \dots \cup P_k = P_1 \oplus P_2 \oplus \dots \oplus P_k$. Because the operator $\oplus$ is associative, we have

$$\begin{aligned} M \oplus (P_1 \cup P_2 \cup \dots \cup P_k) &= M \oplus (P_1 \oplus P_2 \oplus \dots \oplus P_k) \\ &= (\dots ((M \oplus P_1) \oplus P_2) \oplus \dots \oplus P_{k-1}) \oplus P_k \end{aligned}$$

P_k .

A simple induction on i using Lemma 25.1 shows that $M \oplus (P_1 \cup P_2 \cup \dots \cup P_{i-1})$ is a matching in G containing $|M| + i - 1$ edges and that path P_i is an augmenting path with respect to $M \oplus (P_1 \cup P_2 \cup \dots \cup P_{i-1})$. Each of these augmenting paths increases the size of the matching by 1, and so $|M'| = |M| + k$. ■

As the Hopcroft-Karp algorithm goes from matching to matching, it will be useful to consider the symmetric difference between two matchings.

Lemma 25.3

Let M and M^* be matchings in graph $G = (V, E)$, and consider the graph $G' = (V, E')$, where $E' = M \oplus M^*$. Then, G' is a disjoint union of simple paths, simple cycles, and/or isolated vertices. The edges in each such simple path or simple cycle alternate between M and M^* . If $|M^*| > |M|$, then G' contains at least $|M^*| - |M|$ vertex-disjoint M -augmenting paths.

Proof Each vertex in G' has degree 0, 1, or 2, since at most two edges of E' can be incident on a vertex: at most one edge from M and at most one edge from M^* . Therefore, each connected component of G' is either a singleton vertex, an even-length simple cycle with edges alternately in M and M^* , or a simple path with edges alternately in M and M^* . Since

$$\begin{aligned} E' &= M \oplus M^* \\ &= (M \cup M^*) - (M \cap M^*) \end{aligned}$$

and $|M^*| > |M|$, the edge set E' must contain $|M^*| - |M|$ more edges from M^* than from M . Because each cycle in G' has an even number of edges drawn alternately from M and M^* , each cycle has an equal number of edges from M and M^* . Therefore, the simple paths in G' account for there being $|M^*| - |M|$ more edges from M^* than M . Each path containing a different number of edges from M and M^* either starts and ends with edges from M , containing one more edge from M

than from M^* , or starts and ends with edges from M^* , containing one more edge from M^* than from M . Because E' contains $|M^*| - |M|$ more edges from M^* than from M , there are at least $|M^*| - |M|$ paths of the latter type, and each one is an M -augmenting path. Because each vertex has at most two incident edges from E' , these paths must be vertex-disjoint. ■

If an algorithm finds a maximum matching by incrementally increasing the size of the matching, how does it determine when to stop? The following corollary gives the answer: when there are no augmenting paths.

Corollary 25.4

Matching M in graph $G = (V, E)$ is a maximum matching if and only if G contains no M -augmenting path.

Proof We prove the contrapositive of both directions of the lemma statement. The contrapositive of the forward direction is straightforward. If there is an M -augmenting path P in G , then by Lemma 25.1, the matching $M \oplus P$ contains one more edge than M , meaning that M could not be a maximum matching.

To show the contrapositive of the backward direction—if M is not a maximum matching, then G contains an M -augmenting path—let M^* be a maximum matching in Lemma 25.3, so that $|M^*| > |M|$. Then G contains at least $|M^*| - |M| > 0$ vertex-disjoint M -augmenting paths. ■

We already have learned enough to create a maximum-matching algorithm that runs in $O(VE)$ time. Start with the matching M empty. Then repeatedly run a variant of either breadth-first search or depth-first search from an unmatched vertex that takes alternating paths until you find another unmatched vertex. Use the resulting M -augmenting path to increase the size of M by 1.

The Hopcroft-Karp algorithm

The Hopcroft-Karp algorithm improves the running time to $O(\sqrt{V}E)$. The procedure HOPCROFT-KARP is given an undirected bipartite graph, and it uses Corollary 25.2 to repeatedly increase the size of the matching M it finds. Corollary 25.4 proves that the algorithm is correct, since it terminates once there are no M -augmenting paths. It remains to show that the algorithm does run in $O(\sqrt{V}E)$ time. We'll see that the **repeat** loop of lines 2–5 iterates $O(\sqrt{V})$ times and how to implement line 3 so that it runs in $O(E)$ time in each iteration.

HOPCROFT-KARP (G)

```

1  $M = \emptyset$ 
2 repeat
3   let  $\mathcal{P} = \{P_1, P_2, \dots, P_k\}$  be a maximal set of vertex-disjoint
      shortest  $M$ -augmenting paths
4    $M = M \oplus (P_1 \cup P_2 \cup \dots \cup P_k)$ 
5 until  $\mathcal{P} == \emptyset$ 
6 return  $M$ 
```

Let's first see how to find a maximal set of vertex-disjoint shortest M -augmenting paths in $O(E)$ time. There are three phases. The first phase forms a directed version G_M of the undirected bipartite graph G . The second phase creates a directed acyclic graph H from G_M via a variant of breadth-first search. The third phase finds a maximal set of vertex-disjoint shortest M -augmenting paths by running a variant of depth-first search on the transpose H^T of H . (Recall that the transpose of a directed graph reverses the direction of each edge. Since H is acyclic, so is H^T .)

Given a matching M , you can think of an M -augmenting path P as starting at an unmatched vertex in L , traversing an odd number of edges, and ending at an unmatched vertex in R . The edges in P traversed from L to R must belong to $E - M$, and the edges in P traversed from R to L must belong to M . The first phase, therefore,

creates the directed graph G_M by directing the edges accordingly: $G_M = (V, E_M)$, where

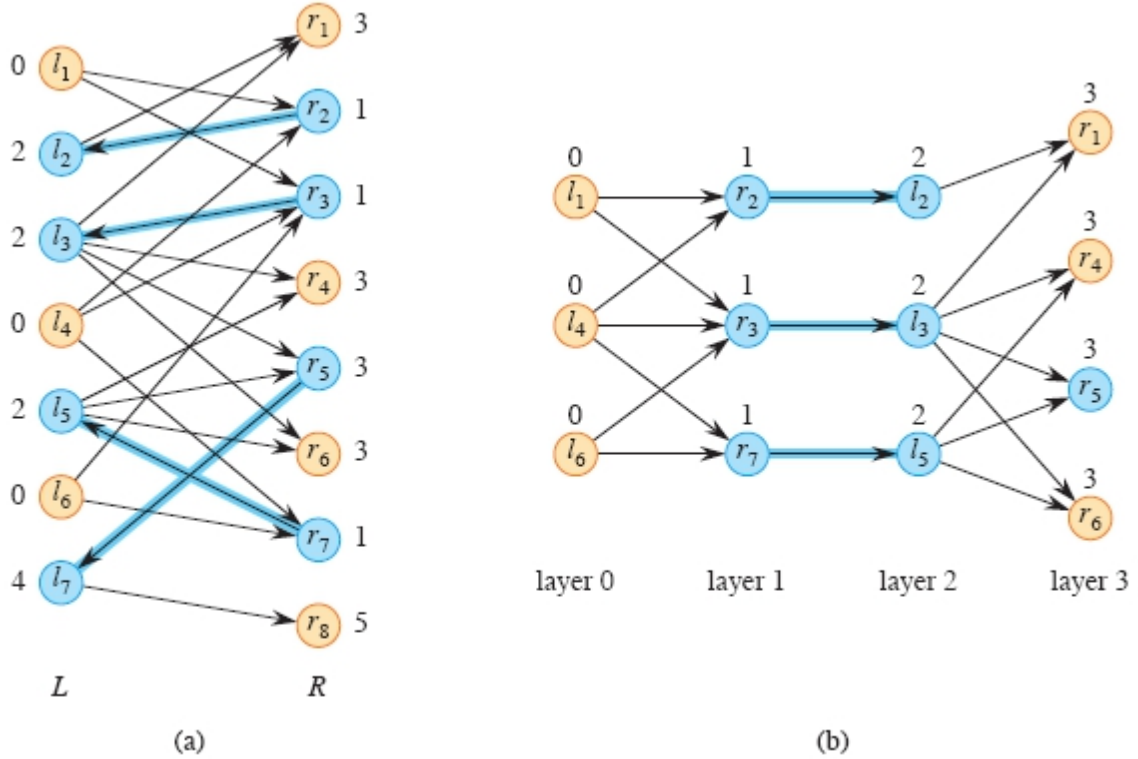


Figure 25.2 (a) The directed graph G_M created in the first phase for the undirected bipartite graph G and matching M in Figure 25.1(a). Breadth-first distances from any unmatched vertex in L appear next to each vertex. (b) The dag H created from G_M in the second phase. Because the smallest distance to an unmatched vertex in R is 3, vertices l_7 and r_8 , with distances greater than 3, are not in H .

$$E_M = \{(l, r) : l \in L, r \in R, \text{ and } (l, r) \in E - M\} \text{ (edges from } L \text{ to } R) \\ \cup \{(r, l) : r \in R, l \in L, \text{ and } (l, r) \in M\} \text{ (edges from } R \text{ to } L).$$

Figure 25.2(a) shows the graph G_M for the graph G and matching M in Figure 25.1(a).

The dag $H = (V_H, E_H)$ created by the second phase has layers of vertices. Figure 25.2(b) shows the dag H corresponding to the directed graph G_M in part (a) of the figure. Each layer contains only vertices

from L or only vertices from R , alternating from layer to layer. The layer that a vertex resides in is given by that vertex's minimum breadth-first distance in G_M from any unmatched vertex in L . Vertices in L appear in even-numbered layers, and vertices in R appear in odd-numbered layers. Let q denote the smallest distance in G_M of any unmatched vertex in R . Then, the last layer in H contains the vertices in R with distance q . Vertices whose distance exceeds q do not appear in V_H . (The graph H in Figure 25.2(b) omits vertices l_7 and r_8 because their distances from any unmatched vertex in L exceed $q = 3$.) The edges in E_H form a subset of E_M :

$$E_H = \{(l, r) \in E_M : r.d \leq q \text{ and } r.d = l.d + 1\} \cup \{(r, l) \in E_M : l.d \leq q\},$$

where the attribute d of a vertex gives the vertex's breadth-first distance in G_M from any unmatched vertex in L . Edges that do not go between two consecutive layers are omitted from E_H .

To determine the breadth-first distances of vertices, run breadth-first search on the graph G_M , but starting from all the unmatched vertices in L . (In the BFS procedure on page 556, replace the root vertex s by the set of unmatched vertices in L .) The predecessor attributes π computed by the BFS procedure are not needed here, since H is a dag and not necessarily a tree.

Every path in H from a vertex in layer 0 to an unmatched vertex in layer q corresponds to a shortest M -augmenting path in the original bipartite graph G . Just use the undirected versions of the directed edges in H . Moreover, every shortest M -augmenting path in G is present in H .

The third phase identifies a maximal set of vertex-disjoint shortest M -augmenting paths. As Figure 25.3 shows, it starts by creating the transpose H^T of H . Then, for each unmatched vertex r in layer q , it performs a depth-first search starting from r until it either reaches a vertex in layer 0 or has exhausted all possible paths without reaching a vertex in layer 0. Instead of maintaining discovery and finish times, the depth-first search just needs to keep track of the predecessor attributes π in the depth-first tree of each search. Upon reaching a vertex in layer 0, tracing back along the predecessors identifies an M -augmenting

path. Each vertex is searched from only when it is first discovered in any search. If the search from a vertex r in layer q cannot find a path of undiscovered vertices to an undiscovered vertex in layer 0, then no M -augmenting path including r goes into the maximal set.

Figure 25.3 shows the result of the third phase. The first depth-first search starts from vertex r_1 . It identifies the M -augmenting path $\langle (r_1, l_3), (l_3, r_3), (r_3, l_1) \rangle$, which is highlighted in orange, and discovers vertices r_1, l_3, r_3 , and l_1 . The next depth-first search starts from vertex r_4 . This search first examines the edge (r_4, l_3) , but because l_3 was already discovered, it backtracks and examines edge (r_4, l_5) . From there, it continues and identifies the M -augmenting path $\langle (r_4, l_5), (l_5, r_7), (r_7, l_6) \rangle$, which is highlighted in yellow, and discovers vertices r_4, l_5, r_7 , and l_6 . The depth-first search from vertex r_6 gets stuck at vertices l_3 and l_5 , which have already been discovered, and so this search fails to find a path of undiscovered vertices to a vertex in layer 0. There is no depth-first search from vertex r_5 because it is matched, and depth-first searches start from unmatched vertices. Therefore, the maximal set of vertex-disjoint shortest M -augmenting paths found contains just the two M -augmenting paths $\langle (r_1, l_3), (l_3, r_3), (r_3, l_1) \rangle$ and $\langle (r_4, l_5), (l_5, r_7), (r_7, l_6) \rangle$.

You might have noticed that in this example, this maximal set of two vertex-disjoint shortest M -augmenting paths is not a maximum set. The graph contains three vertex-disjoint shortest M -augmenting paths: $\langle (r_1, l_2), (l_2, r_2), (r_2, l_1) \rangle$, $\langle (r_4, l_3), (l_3, r_3), (r_3, l_4) \rangle$, and $\langle (r_6, l_5), (l_5, r_7), (r_7, l_6) \rangle$. No matter: the algorithm requires the set of vertex-disjoint shortest M -augmenting paths found in line 3 of HOPCROFT-KARP to be only maximal, not necessarily maximum.

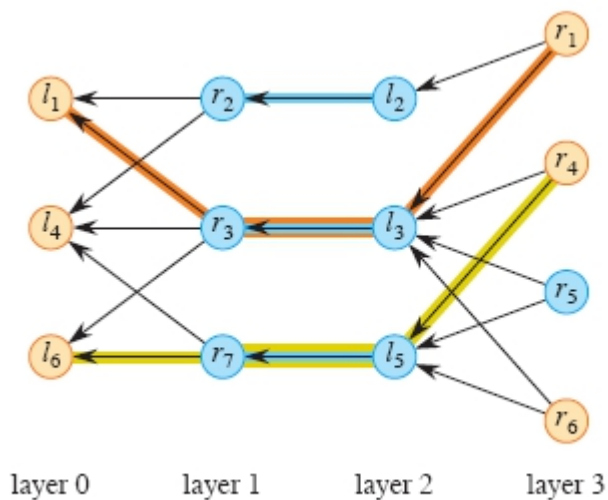


Figure 25.3 The transpose H^T of the dag H created in the third phase. The first depth-first search, starting from vertex r_1 , identifies the M -augmenting path $\langle (r_1, l_3), (l_3, r_3), (r_3, l_1) \rangle$ highlighted in orange, and it discovers vertices r_1, l_3, r_3, l_1 . The second depth-first search, starting from vertex r_4 , identifies the M -augmenting path $\langle (r_4, l_5), (l_5, r_7), (r_7, l_6) \rangle$ highlighted in yellow, discovering vertices r_4, l_5, r_7, l_6 .

It remains to show that all three phases of line 3 take $O(E)$ time. We assume that in the original bipartite graph G , each vertex has at least one incident edge so that $|V| = O(E)$, which in turn implies that $|V| + |E| = O(E)$. The first phase creates the directed graph G_M by simply directing each edge of G , so that $|V_M| = |V|$ and $|E_M| = |E|$. The second phase performs a breadth-first search on G_M , taking $O(V_M + E_M) = O(E_M) = O(E)$ time. In fact, it can stop once the first distance in the queue within the breadth-first search exceeds the shortest distance q to an unmatched vertex in R . The dag H has $|V_H| \leq |V_M|$ and $|E_H| \leq |E_M|$, so that it takes $O(V_H + E_H) = O(E)$ time to construct. Finally, the third phase performs depth-first searches from the unmatched vertices in layer q . Once a vertex is discovered, it is not searched from again, and so the analysis of depth-first search from Section 20.3 applies here: $O(V_H + E_H) = O(E)$. Hence, all three phases take just $O(E)$ time.

Once the maximal set of vertex-disjoint shortest M -augmenting paths have been found in line 3, updating the matching in line 4 takes $O(E)$ time, as it is just a matter of going through the edges of the M -augmenting paths and adding edges to and removing edges from the matching M . Thus, each iteration of the **repeat** loop of lines 2–5 can run in $O(E)$ time.

It remains to show that the **repeat** loop iterates $O(\sqrt{V})$ times. We start with the following lemma, which shows that after each iteration of the **repeat** loop, the length of an augmenting path increases.

Lemma 25.5

Let $G = (V, E)$ be an undirected bipartite graph with matching M , and let q be the length of a shortest M -augmenting path. Let $\mathcal{P} = \{P_1, P_2, \dots, P_k\}$ be a maximal set of vertex-disjoint M -augmenting paths of length q . Let $M' = M \oplus (P_1 \cup P_2 \cup \dots \cup P_k)$, and suppose that P is a shortest M' -augmenting path. Then P has more than q edges.

Proof We consider separately the cases in which P is vertex-disjoint from the augmenting paths in $\mathcal{P}$ and in which it is not vertex-disjoint.

First, assume that P is vertex-disjoint from the augmenting paths in $\mathcal{P}$. Then, P contains edges that are in M but are not in any of $P_1, P_2, \dots, P_k$, so that P is also an M -augmenting path. Since P is disjoint from $P_1, P_2, \dots, P_k$ but is also an M -augmenting path, and since $\mathcal{P}$ is a maximal set of shortest M -augmenting paths, P must be longer than any of the augmenting paths in $\mathcal{P}$, each of which has length q . Therefore, P has more than q edges.

Now, assume that P visits at least one vertex from the M -augmenting paths in $\mathcal{P}$. By Corollary 25.2, M' is a matching in G with $|M'| = |M| + k$. Since P is an M' -augmenting path, by Lemma 25.1, $M' \oplus P$ is a matching with $|M' \oplus P| = |M'| + 1 = |M| + k + 1$. Now let $A = M \oplus M' \oplus P$. We claim that $A = (P_1 \cup P_2 \cup \dots \cup P_k) \oplus P$:

$$\begin{aligned} A &= M \oplus M' \oplus P \\ &= M \oplus (M \oplus (P_1 \cup P_2 \cup \dots \cup P_k)) \oplus P \end{aligned}$$

$$\begin{aligned}
&= (M \oplus M) \oplus (P_1 \cup P_2 \cup \cdots \cup P_k) \oplus P \text{ (associativity of } \oplus) \\
&= \emptyset \oplus (P_1 \cup P_2 \cup \cdots \cup P_k) \oplus P \quad (X \oplus X = \emptyset \text{ for all } X) \\
&= (P_1 \cup P_2 \cup \cdots \cup P_k) \oplus P \quad (\emptyset \oplus X = X \text{ for all } X).
\end{aligned}$$

Lemma 25.3 with $M^* = M' \oplus P$ gives that A contains at least $|M' \oplus P| - |M| = k + 1$ vertex-disjoint M -augmenting paths. Since each such M -augmenting path has at least q edges, we have $|A| \geq (k + 1)q = kq + q$.

Now we claim that P shares at least one edge with some M -augmenting path in $\mathcal{P}$. Under the matching M' , every vertex in each M -augmenting path in $\mathcal{P}$ is matched. (Only the first and last vertex in each M -augmenting path P_i is unmatched under M , and under $M \oplus P_i$, all vertices in P_i are matched. Because the M -augmenting paths in $\mathcal{P}$ are vertex-disjoint, no other path in $\mathcal{P}$ can affect whether the vertices in P_i are matched. That is, the vertices in P_i are matched under $(M \oplus P_i) \oplus P_j$ if and only if they are matched under $M \oplus P_i$, for any other path $P_j \in \mathcal{P}$.) Suppose that P shares a vertex v with some path $P_i \in \mathcal{P}$. Vertex v cannot be an endpoint of P , because the endpoints of P are unmatched under M' . Therefore, v has an incident edge in P that belongs to M' . Since any vertex has at most one incident edge in a matching, this edge must also belong to P_i , thus proving the claim.

Because $A = (P_1 \cup P_2 \cup \cdots \cup P_k) \oplus P$ and P shares at least one edge with some $P_i \in \mathcal{P}$, we have that $|A| < |P_1 \cup P_2 \cup \cdots \cup P_k| + |P|$. Thus, we have

$$\begin{aligned}
kq + q &\leq |A| \\
&< |P_1 \cup P_2 \cup \cdots \cup P_k| + |P| \\
&= kq + |P|,
\end{aligned}$$

so that $q < |P|$. We conclude that P contains more than q edges. ■

The next lemma bounds the size of a maximum matching, based on the length of a shortest augmenting path.

Lemma 25.6

Let M be a matching in graph $G = (V, E)$, and let a shortest M -augmenting path in G contain q edges. Then the size of a maximum matching in G is at most $|M| + |V| / (q + 1)$.

Proof Let M^* be a maximum matching in G . By Lemma 25.3, G contains at least $|M^*| - |M|$ vertex-disjoint M -augmenting paths. Each of these paths contains at least q edges, and hence at least $q + 1$ vertices. Because these paths are vertex-disjoint, we have $(|M^*| - |M|)(q + 1) \leq |V|$, so that $|M^*| \leq |M| + |V| / (q + 1)$. ■

The final lemma bounds the number of iterations of the **repeat** loop of lines 2–5.

Lemma 25.7

When the HOPCROFT-KARP procedure runs on an undirected bipartite graph $G = (V, E)$, the **repeat** loop of lines 2–5 iterates $O(\sqrt{V})$ times.

Proof By Lemma 25.5, the length q of the shortest M -augmenting paths found in line 3 increases from iteration to iteration. After $\lceil \sqrt{|V|} \rceil$ iterations, therefore, we must have $q \geq \lceil \sqrt{|V|} \rceil$. Consider the situation after the first time line 4 executes with M -augmenting paths whose length is at least $\lceil \sqrt{|V|} \rceil$. Since the size of a matching increases by at least one edge per iteration, Lemma 25.6 implies that the number of additional iterations before achieving a maximum matching is at most

$$\frac{|V|}{\lceil \sqrt{|V|} \rceil + 1} < \frac{|V|}{\sqrt{|V|}} = \sqrt{|V|}.$$

Hence, the total number of loop iterations is less than $2\sqrt{|V|}$. ■

Thus, we have the following bound on the running time of the HOPCROFT-KARP procedure.

Theorem 25.8

The procedure HOPCROFT-KARP runs in $O(\sqrt{V}E)$ time on an undirected bipartite graph $G = (V, E)$.

Proof By Lemma 25.7 the **repeat** loop iterates $O(\sqrt{V})$ times, and we have seen how to implement each iteration in $O(E)$ time. ■

Exercises

25.1-1

Use the Hopcroft-Karp algorithm to find a maximum matching for the graph in Figure 25.1.

25.1-2

How are M -augmenting paths and augmenting paths in flow networks similar? How do they differ?

25.1-3

What is the advantage of searching in the transpose H^T from unmatched vertices in layer q (the first layer that contains an unmatched vertex in R) to layer 0 versus searching in the dag H from layer 0 to layer q ?

25.1-4

Show how to bound the number of iterations of the the **repeat** loop of lines 2–5 of HOPCROFT-KARP by $\lceil 3\sqrt{|V|/2} \rceil$.

★ 25.1-5

A **perfect matching** is a matching under which every vertex is matched. Let $G = (V, E)$ be an undirected bipartite graph with vertex partition $V = L \cup R$, where $|L| = |R|$. For any $X \subseteq V$, define the **neighborhood** of X as $N(X) = \{y \in V : (x, y) \in E \text{ for some } x \in X\}$,

that is, the set of vertices adjacent to some member of X . Prove **Hall's theorem**: there exists a perfect matching in G if and only if $|A| \leq |N(A)|$ for every subset $A \subseteq L$.

25.1-6

In a *d-regular* graph, every vertex has degree d . If $G = (V, E)$ is bipartite with vertex partition $V = L \cup R$ and also d -regular, then $|L| = |R|$. Use Hall's theorem (see Exercise 25.1-5) to prove that every d -regular bipartite graph contains a perfect matching. Then use that result to prove that every d -regular bipartite graph contains d disjoint perfect matchings.

25.2 The stable-marriage problem

In Section 25.1, the goal was to find a maximum matching in an undirected bipartite graph. If you know that the graph $G = (V, E)$ with vertex partition $V = L \cup R$ is a *complete bipartite graph*¹—containing an edge from every vertex in L to every vertex in R —then you can find a maximum matching by a simple greedy algorithm.

When a graph can have several matchings, you might want to decide which matchings are most desirable. In Section 25.3, we'll add weights to the edges and find a matching of maximum weight. In this section, we will instead add some information to each vertex in a complete bipartite graph: a ranking of the vertices in the other side. That is, each vertex in L has an ordered list of all the vertices in R , and vice-versa. To keep things simple, let's assume that L and R each contain n vertices. The goal here is to match each vertex in L with a vertex in R in a “stable” way.

This problem derives its name, the *stable-marriage problem*, from the notion of heterosexual marriage, viewing L as a set of women and R as a set of men.² Each woman ranks all the men in terms of desirability, and each man does the same with all the women. The goal is to pair up women and men (a matching) so that if a woman and a man are not matched to each other, then at least one of them prefers their assigned partner.

If a woman and a man are not matched to each other but each prefers the other over their assigned partner, they form a *blocking pair*. A blocking pair has incentive to opt out of the assigned pairing and get